

PyNumDiff: Practical Numerical Differentiation for Noisy Data

Pavel Komarov¹, Floris van Breugel², Maria Protogerou³, and J. Nathan Kutz⁴

¹ Department of Electrical and Computer Engineering, University of Washington, USA ² Department of Mechanical Engineering, University of Nevada, Reno, USA ³ Department of Applied Mathematics, University of Washington, USA ⁴ Autodesk Research, London, UK ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Prashant Jha](#)

Reviewers:

- [@alexzerg](#)
- [@sidxz](#)

Submitted: 02 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

Derivatives of measured data are vital across science and engineering: identifying governing equations, designing controllers, and processing sensor streams alike. The textbook remedy, finite differencing, amplifies noise as $1/\Delta t$ and deteriorates rapidly as data grows noisier. Smoothing before differencing helps, but algorithm choice and settings substantially affect the result, and no single approach wins universally.

PyNumDiff is an open-source Python package consolidating a broad suite of numerical differentiation methods under a unified API, along with an optimization scheme to select hyperparameters even in the absence of ground truth. It implements seven algorithm families: (1) prefiltering followed by finite difference calculation, (2) iterated finite differencing, (3) polynomial fitting (Savitzky & Golay, 1964), (4) global (spectral) and local (wavelet, radial) basis function fitting, (5) total variation regularization (Chartrand, 2011), (6) Kalman smoothing (Kalman, 1960; Rauch et al., 1965), and (7) local approximation with linear models. All PyNumDiff methods return estimates of the true signal and its derivative as a matched pair, $(\hat{x}, \hat{dx/dt})$. A companion survey (Komarov et al., 2025, sec. 7) situates these methods in the literature, covers the theory behind each method, benchmarks all methods across test signals, and guides selection for different application scenarios. This paper describes the package's second generation, through the v0.3 release series.

Statement of Need

Numerical differentiation has a diverse ecosystem of algorithms, each with different theoretical foundations and putative strengths. But there has been no consolidated home where methods can be easily swapped or tested head-to-head, and practitioners are left assembling solutions piecemeal from disparate packages (van Breugel et al., 2022).

PyNumDiff addresses this gap: Its unified interface lets users compare methods on the same data, exploit specialized capabilities (e.g. outlier robustness), and tune methods' hyperparameters against known ground truth or to a given signal bandlimit. Derivative estimation trades data fidelity against smoothness, so PyNumDiff uses dual-objective optimization, with a single scalar weight to steer the balance (van Breugel et al., 2020). A natural and growing application is SINDy (Brunton et al., 2016), which discovers governing equations by regressing measured derivatives, making quality of the derivative estimates a direct determinant of model accuracy.

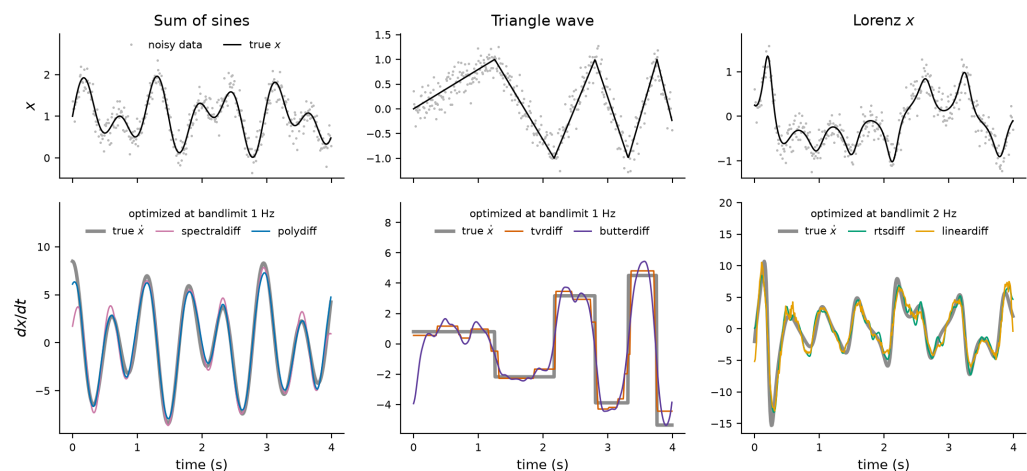


Figure 1: Three simulated signals with additive noise (top) and their derivatives (bottom). Each panel contrasts two methods from different families, six of the seven in total, with a fixed color per method. Hyperparameters come from `pynumdiff.optimize` rather than hand-tuning, at the bandlimit noted with each derivative panel: 1 Hz for the sines and triangle, 2 Hz for the faster Lorenz signal.

State of the Field

Relevant Python tools exist, but each covers only part of the space. `numpy.gradient` and `scipy.signal.savgol_filter` (Virtanen et al., 2020) cover only a couple specific cases; `findiff` (Baer, 2018) offers high-order finite difference stencils suited to clean simulation data, but not to noisy measurements; `pykalman` (Duckworth & The pykalman developers, 2012) does not provide state-space models to turn the filter into a differentiator; and total variation regularization (Chartrand, 2011) was implemented with lagged-diffusivity iteration that approaches the convex minimizer rather than attaining it. Historically, practitioners have had to patch together packages, standalone scripts, and academic mathematics, with no shared API or principled way to compare results. The derivative package (Goldschmidt, 2021) attempts to bring more algorithms under one roof but lacks NaN handling and hyperparameter optimization. No existing package spans PyNumDiff's seven method families with a consistent interface.

The original PyNumDiff publication (van Breugel et al., 2022) established the core method set and optimization framework. This second generation rewrites and consolidates it from the ground up. `kerndiff`, `rtsdiff`, and `tvrdiff` each cover by parameter what had been separate functions, so the package presents a simplified interface. `rbfdiff`, `waveletdiff`, and `robustdiff` are new, bringing the set to thirteen, all organized into seven modules and callable with consistent keyword-argument signatures. A single optimizer serves every one of them. This generation also adds support for multidimensional data, as well as irregular sample spacing, missing observations, and circular domains where the underlying mathematics allows.

Software Design

Package design. All differentiation methods share the call signature

```
x_hat, dxdt_hat = method(x, dt_or_t, **hyperparams)
```

where `x` is a NumPy array (Harris et al., 2020) of measurements, `dt_or_t` is either a scalar step size or an array of sample locations, and keyword arguments configure the method, making calls self-documenting. The v0.2 series preserves prior positional signatures behind deprecation warnings; v0.3 removes them, leaving only the keyword-argument interface.

Architecture. PyNumDiff is organized into seven differentiation modules plus shared utils and optimize in a flat structure. Where strong alternatives exist, PyNumDiff delegates rather than reimplements: SciPy (Virtanen et al., 2020) provides spline fitting, Savitzky-Golay filtering, and signal processing routines; NumPy (Harris et al., 2020) provides the FFT; PyWavelets (Lee et al., 2019) provides the discrete wavelet transform for wavediff; CVXPY (Diamond & Boyd, 2016) handles convex optimization for robustdiff, tvrdiff, and lineardiff. The public kalman_filter and rts_smooth primitives let advanced users with known dynamics bypass rtsdiff's constant-derivative model to build their own bespoke differentiator. utils houses common subroutines like estimation of integration constants and data scales, calculation of evaluation metrics, and generation of simulated noisy data. optimize includes not only cost minimization but a whole framework: per-method default search spaces, bounds with rounding rules, categorical handling, a caching layer, and a configurable objective that can work even in the absence of ground-truth.

Method capabilities. Every method handles multidimensional data via axis. Table 1 groups methods by their additional talents.

Capability	Methods
Variable step size	polydiff, splinediff, rbfdiff, rtsdiff, robustdiff, lineardiff
Missing data	polydiff, splinediff, rtsdiff, robustdiff, lineardiff
Outlier robustness	robustdiff, tvrdiff
Circular domain	rtsdiff

Table 1: Methods by auxiliary capability.

Irregular and incomplete sampling. Whether and how a method supports variable step size depends on the underlying model. For example, basis spline fits are indifferent to spacing, able to place knots with equal ease anywhere in the domain, but Kalman-based methods must compute a discrete-time transition by matrix exponential at each actual interval, while Butterworth filters inflexibly assume a constant sampling rate. Likewise, missing observations, given as NaN entries, can be excluded from fitting and imputed from the model in select cases, useful for real data where sensors may drop samples.

Outlier robustness. robustdiff replaces the quadratic Kalman cost with Huber loss (Huber, 1964) terms on both measurement and process residuals, following Aravkin et al. (2013), with CVXPY (Diamond & Boyd, 2016) as the optimization backend, operating over a sparse problem formulation to scale linearly with signal length. tvrdiff similarly applies Huber loss on data fidelity, with its total variation penalty on the derivative additionally promoting piecewise-smooth solutions.

Circular and wrapped domains. rtsdiff accepts circular=True for quantities like angles on a periodic domain. Innovation residuals are wrapped to $[-\pi, \pi]$ before each Kalman update via an innovation_fn hook, and x_hat is returned in the same range, avoiding the large spurious spikes naive smoothers produce when a signal crosses the $\pm\pi$ boundary.

Hyperparameter optimization. pynumdiff.optimize selects hyperparameters Φ by minimizing

$$L(\Phi) = \text{RMSE}(\int \hat{x}(\Phi) + c, x) + \gamma \text{TV}(\hat{x}(\Phi)),$$

which requires no ground truth, because fidelity is measured by integrating the estimated derivative back against the measured signal (van Breugel et al., 2020). Root mean squared error in the first term can be (and is by default since v0.2.1) replaced with a robust variant, so it is less prone to bias in the presence of outliers. The smoothness weight γ is derived internally from the data's sampling rate and user-provided bandlimit, the highest frequency of meaningful signal. Default search spaces are defined at the top of the optimize module, with opportunities to collapse to fewer search dimensions thoroughly explored, for speed. Minimization requires a continuous space, so discrete numbers like polynomial order and

106 window size are handled by rounding, while truly categorical hyperparameters like kernel type
107 are supported by pooling results across runs employing separate choices. Repeated evaluations
108 are cached to avoid redundant work. A single call to `suggest_method` runs this search across
109 nearly every method in the package and reports the best fit, the practical payoff of putting all
110 behind one interface.

111 **Testing and continuous integration.** The test suite validates all methods against analytic
112 functions with known derivatives, covering noiseless and noisy cases, scale equivariance,
113 multidimensional application, and missing-data handling. Utilities and optimization are also
114 tested for correct behavior, to verify functionality like robust estimation of data scale and
115 the equivalence of parallel and serial search. The tests avoid tautology, never letting an
116 implementation define its own expected result: Accuracy is asserted against empirically
117 established error bounds, with the suite reporting any method that beats its bound by an order
118 of magnitude so the bound can be ratcheted down. An optional `--plot` flag renders every
119 case for visual inspection. GitHub Actions runs the suite on every push and pull request, and
120 Coveralls tracks line coverage, currently 94%.

121 Research Impact Statement

122 The first generation ([van Breugel et al., 2022](#)) of PyNumDiff is cited in over twenty publications
123 and has been applied in experimental biology (flight kinematics from motion capture) and
124 control engineering (observer design). The PySINDy project ([Silva et al., 2020](#)) maintains its
125 own differentiation submodule overlapping substantially with PyNumDiff's capabilities, and
126 integration discussions are ongoing.

127 The companion taxonomy paper ([Komarov et al., 2025, sec. 7](#)) provides accuracy and
128 bias results for every included method and derives recommendations, based on performance
129 against simulated signals, using loss, search spaces, and other choices now encoded in
130 PyNumDiff v0.3. The evidence is reproducible, with experiments runnable from Jupyter
131 notebooks in PyNumDiff's repo, spanning sampling rate, noise distribution, noise scale,
132 outlier contamination, and signal bandlimit. Additional tutorial notebooks cover basic usage,
133 hyperparameter optimization, automatic method suggestion, and demonstrations of the outlier,
134 multidimensional, and circular-domain capabilities described above. Full API documentation is
135 published at pynumdiff.readthedocs.io.

136 AI Usage Disclosure

137 This paper was drafted with assistance from Claude Opus 5 (Anthropic), which also helped
138 draft code revisions before and during review. All outputs were reviewed and heavily remolded
139 by hand, and the authors take full responsibility for accuracy.

140 Acknowledgements

141 The authors thank Yuying Liu and Bingni W. Brunton for their contributions to the original
142 PyNumDiff package ([van Breugel et al., 2022](#)), and Sasha Aravkin for discussions on convex
143 optimization techniques that informed robust differentiation methods. This work was supported
144 by the NSF AI Institute in Dynamic Systems (grant number 2112085).

145 References

146 Aravkin, A. Y., Burke, J. V., & Pillonetto, G. (2013). Optimization viewpoint on Kalman
147 smoothing with applications to robust and sparse estimation. *Journal of Machine Learning*
148 *Research*, 14, 2513–2558. https://doi.org/10.1007/978-3-642-38398-4_8

- 149 Baer, M. (2018). *Findiff: Finite difference derivatives in Python*. [https://github.com/maroba/](https://github.com/maroba/findiff)
150 [findiff](https://github.com/maroba/findiff)
- 151 Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from
152 data by sparse identification of nonlinear dynamical systems. *Proceedings of the National*
153 *Academy of Sciences*, 113(15), 3932–3937. <https://doi.org/10.1073/pnas.1517384113>
- 154 Chartrand, R. (2011). Numerical differentiation of noisy, nonsmooth data. *ISRN Applied*
155 *Mathematics*, 2011, 164564. <https://doi.org/10.5402/2011/164564>
- 156 Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex
157 optimization. *Journal of Machine Learning Research*, 17(83), 1–5.
- 158 Duckworth, D., & The pykalman developers. (2012). *Pykalman: Kalman filters and smoothers*
159 *for Python*. <https://github.com/pykalman/pykalman>
- 160 Goldschmidt, A. (2021). *Derivative: Numerical differentiation in Python*. [https://github.com/](https://github.com/andgoldschmidt/derivative)
161 [andgoldschmidt/derivative](https://github.com/andgoldschmidt/derivative)
- 162 Harris, C. R., Millman, K. J., van der Walt, S. J., & others. (2020). Array programming with
163 NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 164 Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical*
165 *Statistics*, 35(1), 73–101. <https://doi.org/10.1214/aoms/1177703732>
- 166 Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of*
167 *Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>
- 168 Komarov, P., van Breugel, F., & Kutz, J. N. (2025). *A taxonomy of numerical differentiation*
169 *methods*. <https://arxiv.org/abs/2512.09090>
- 170 Lee, G. R., Gommers, R., Wasilewski, F., Wohlfahrt, K., & O’Leary, A. (2019). PyWavelets:
171 A Python package for wavelet analysis. *Journal of Open Source Software*, 4(36), 1237.
172 <https://doi.org/10.21105/joss.01237>
- 173 Rauch, H. E., Tung, F., & Striebel, C. T. (1965). Maximum likelihood estimates of linear
174 dynamic systems. *AIAA Journal*, 3(8), 1445–1450. <https://doi.org/10.2514/3.3166>
- 175 Savitzky, A., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified
176 least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. [https://doi.org/10.](https://doi.org/10.1021/ac60214a047)
177 [1021/ac60214a047](https://doi.org/10.1021/ac60214a047)
- 178 Silva, B. M. de, Champion, K., Quade, M., Loiseau, J.-C., Kutz, J. N., & Brunton, S. L. (2020).
179 PySINDy: A Python package for the sparse identification of nonlinear dynamics from data.
180 *Journal of Open Source Software*, 5(49), 2104. <https://doi.org/10.21105/joss.02104>
- 181 van Breugel, F., Kutz, J. N., & Brunton, B. W. (2020). Numerical differentiation of noisy
182 data: A unifying multi-objective optimization framework. *IEEE Access*, 8, 196865–196877.
183 <https://doi.org/10.1109/ACCESS.2020.3034077>
- 184 van Breugel, F., Liu, Y., Brunton, B. W., & Kutz, J. N. (2022). PyNumDiff: A Python
185 package for numerical differentiation of noisy time-series data. *Journal of Open Source*
186 *Software*, 7(71), 4078. <https://doi.org/10.21105/joss.04078>
- 187 Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: Fundamental
188 algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
189